

Kilometer-Scale Probabilistic Weather Forecasting over Taiwan via Conditional Flow Matching

講者：鍾鎮嶸

Why this matters: Taiwan's weather

- ~3-5 typhoons make landfall on Taiwan per year; the 2022 record includes multiple events with > 200 mm/h hourly rainfall.
- Flash flooding, landslides on the central mountain range, and agricultural losses are dominated by a few extreme hourly events.
- Civil aviation, marine fishery, and reservoir management all run on hourly km-scale forecasts.

Resolutions and the CFL condition

- Spatial resolution
 - grid spacing / pixel resolution of weather data
 - Two regimes:
 - Synoptic / Global (~10-25 km): resolves cyclones
 - Convection-allowing / regional (~1-4 km): resolves thunderstorms, orographic precipitation
- Temporal resolution
 - time step between two adjacent weather data
- Changes in resolutions has to obey the Courant–Friedrichs–Lewy condition to be mathematically stable

$$C = \left(\frac{|u|}{\Delta x} + \frac{|v|}{\Delta y} + \frac{|w|}{\Delta z} \right) \Delta t \leq C_{\max} \quad \left(C_{\max} \sim 1 \text{ for explicit schemes} \right) \Delta x \downarrow 2\times \implies \Delta t \downarrow 2\times \implies \text{3-D cost} \uparrow 2^4 = 16\times$$

Numerical Weather Prediction's computational requirement

- ECMWF (European Centre for Medium-Range Weather Forecasts)
 - Global operational weather forecasting and massive climate re-analysis (ERA5)
 - CPU: 1,015,808 AMD EPYC Rome CPU cores
 - Memory: 2,050 Tb
 - 30.0 PetaFLOPS (10^{15})
- RIKEN Center for Computational Science
 - "Mega-ensemble" forecasting. (e.g., Running 1,000 simultaneous ensemble members to accurately predict probabilistic events like the July 2020 Kyushu heavy rain)
 - CPU: 7,630,848 Fujitsu A64FX cores
 - Memory: 4,850 Tb
 - 997 PetaFLOPS (10^{15})

The mesoscale bottleneck

- Synoptic scale (~25 km grid, GFS / ECMWF-IFS): cheap, runs every 6 h.
- Convection-allowing models / CAMs (~3 km grid, HRRR / RWRF): must resolve thunderstorms, cold pools, gravity waves
 - $25/3 = 8.333$ times of change in resolution in each dimension
 - To obey CFL, computation is 8.333^3 (3 dimensions) * 8.333 (temporal change) = ~ 4821 more expensive

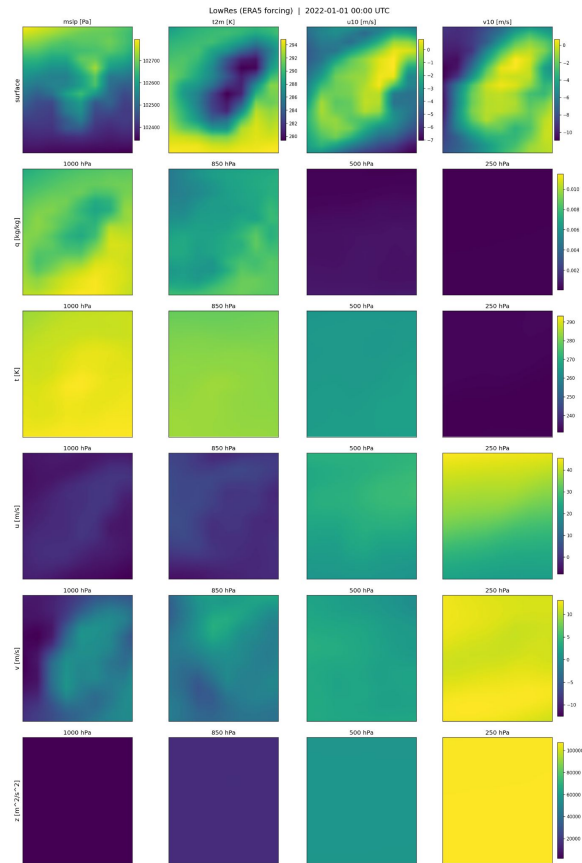
Related work: ML weather forecasting landscape

- Global deterministic:
 - FourCastNet (Nvidia) — AFNO-based $\sim 0.25^\circ$ emulator.
 - Pangu-Weather (Huawei, Nature'23) — 3-D transformer; beats deterministic IFS at 1–7 day leads.
 - GraphCast (Deepmind, Science'23) — GNN, 0.25° , current SOTA on deterministic global skill.
- Global probabilistic:
 - GenCast (Deepmind, Nature'24) — diffusion-based 0.25° ensemble; beats ENS on most headline scores.
- Regional / convection-allowing:
 - StormCast (Nvidia) - 3 km HRRR diffusion
 - CorrDiff (Nvidia) - diffusion super-resolution on Taiwan domain; static (no autoregressive rollout)

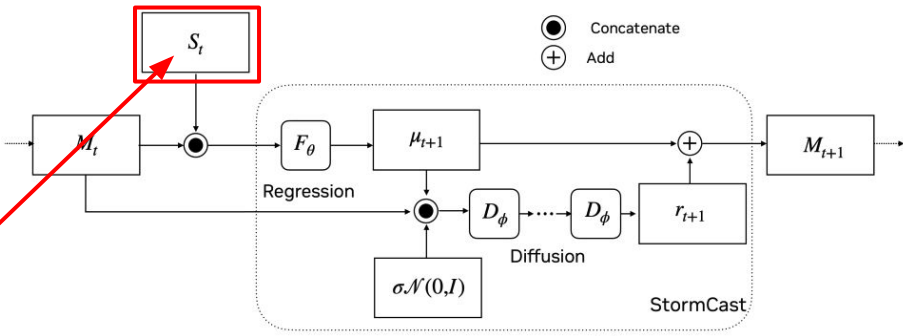
Related work: Flow matching

- Flow matching
- I-CFM / Independent CFM
- FlowCast — CFM for radar nowcasting in latent space

StormCast (Nvidia, 2024), collaboration with NCDR

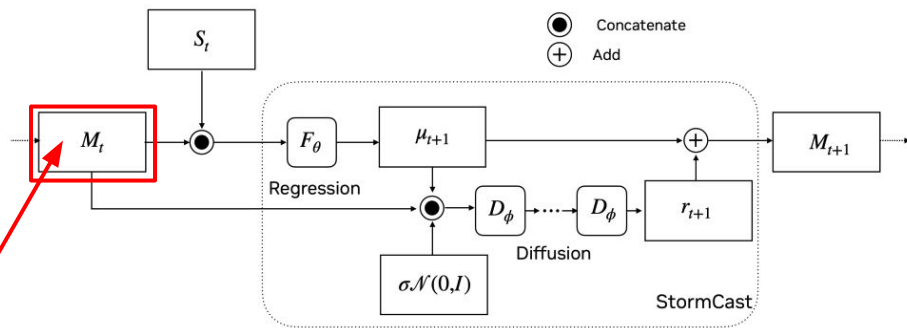
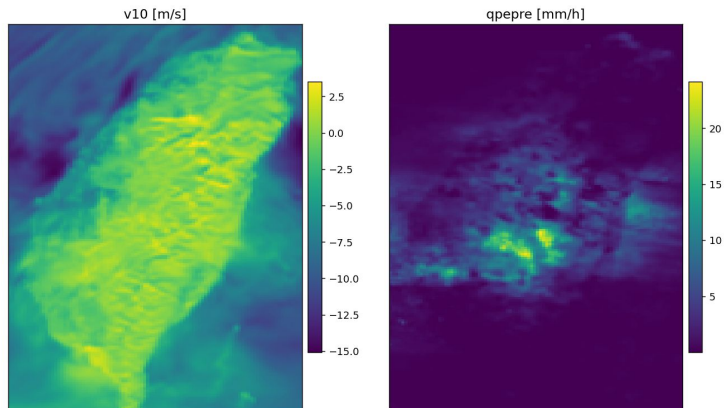
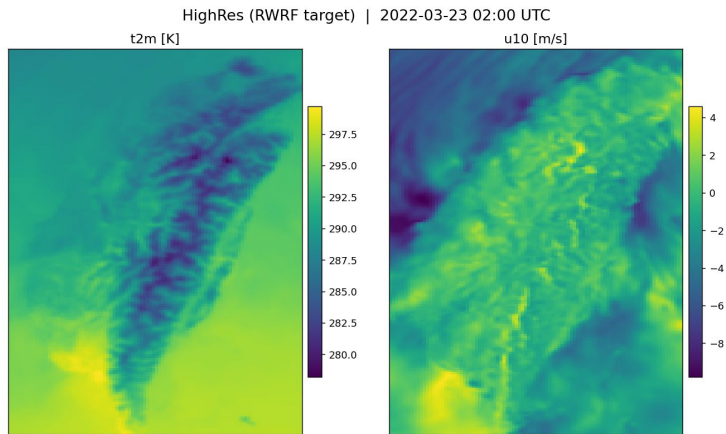


low resolution data:
ERA5, ~25km



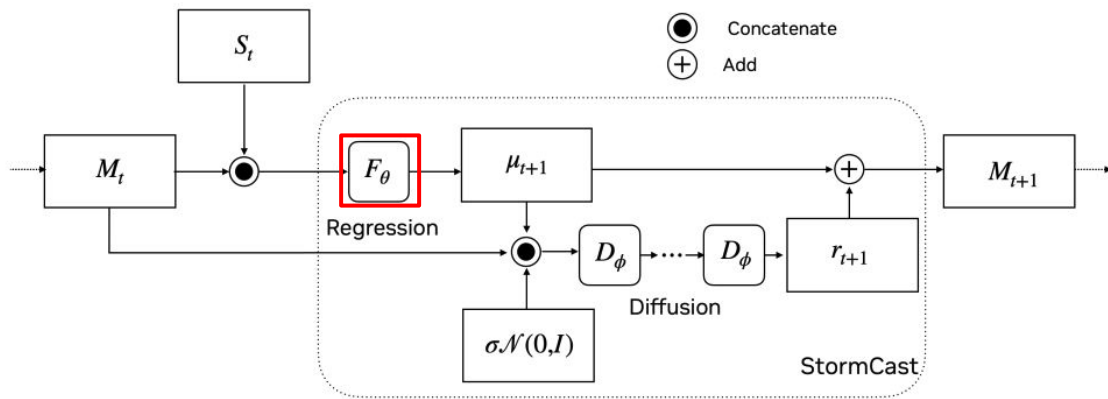
Symbol	Name	Units
<i>Pressure-level variables</i> — sampled at {1000, 850, 500, 250} hPa		
q	Specific humidity	kg/kg
t	Temperature	K
u	Zonal wind	m/s
v	Meridional wind	m/s
z	Geopotential	m^2/s^2
<i>Surface variables</i> (single 2-D field per time step)		
mslp	Mean sea-level pressure	Pa
t2m	2-metre temperature	K
u10	10-metre zonal wind	m/s
v10	10-metre meridional wind	m/s

StormCast (Nvidia, 2024), collaboration with NCDR



high resolution data:
RWRF, ~3km (NWP)

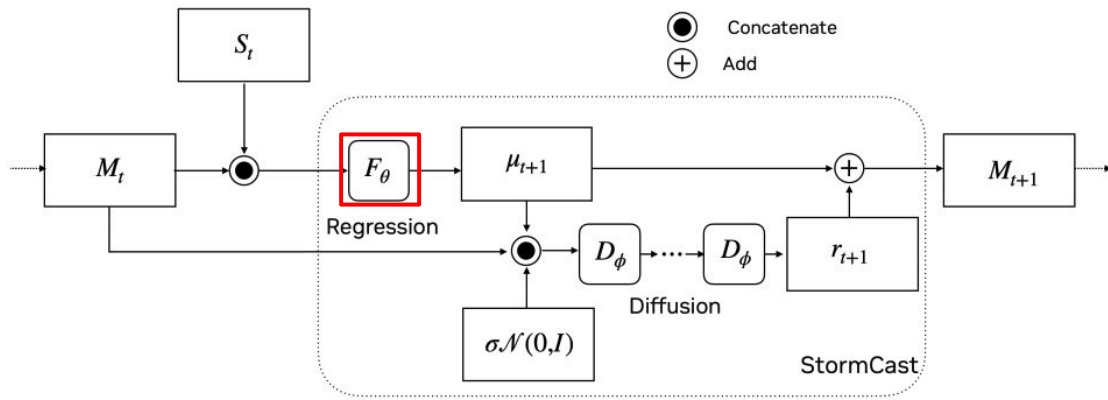
StormCast (Nvidia, 2024), collaboration with NCDR



$$F_\theta(M_t, S_t) = \mathbb{E}[M_{t+1} \mid M_t, S_t]$$

**learn a deterministic regression F_θ
to estimate the conditional mean**

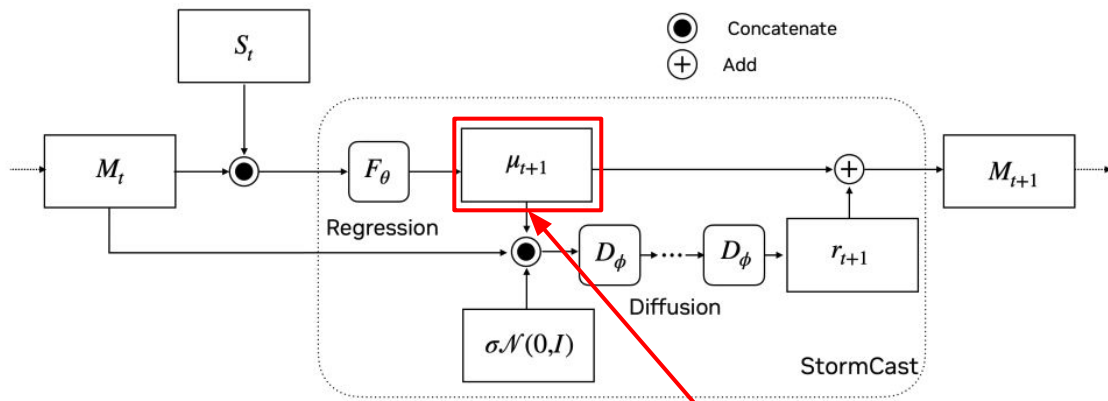
StormCast (Nvidia, 2024), collaboration with NCDR



$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \left\| M_{t_i+1} - F_\theta(M_{t_i}, S_{t_i}) \right\|_2^2$$

train a UNet based on the adjacent paired samples

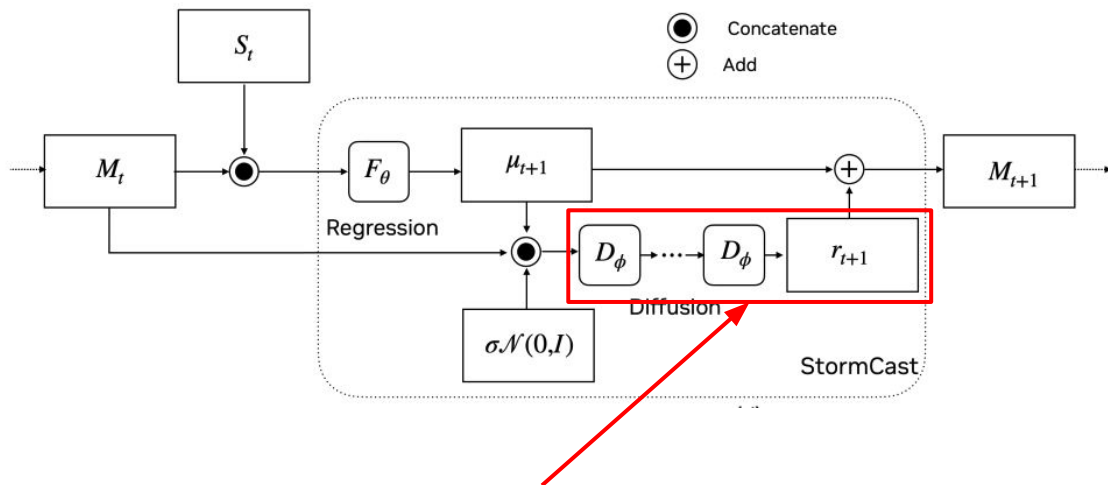
StormCast (Nvidia, 2024), collaboration with NCDR



$$r_{t+1} := M_{t+1} - \mu_{t+1} \quad \text{where} \quad \mu_{t+1} := F_{\theta^*}(M_t, S_t)$$

**with trained regression model,
we can form the residuals**

StormCast (Nvidia, 2024), collaboration with NCDR



$$p(r_{t+1} \mid \mu_{t+1}, M_t, S_t)$$

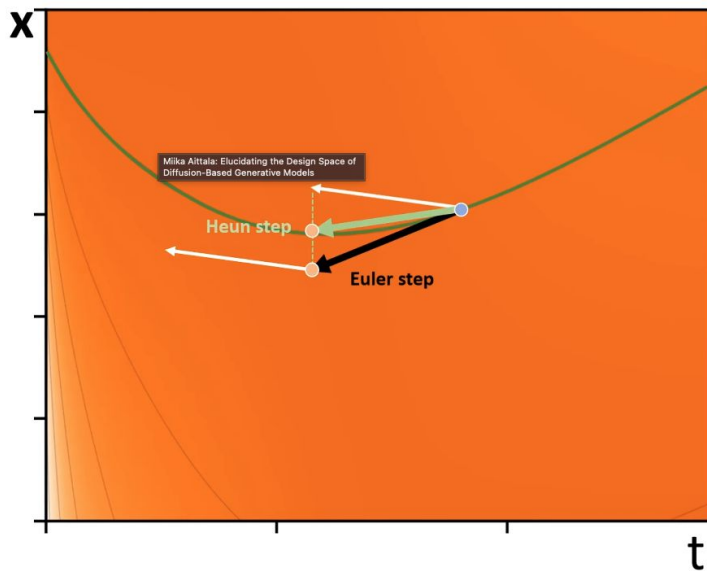
- Diffusion model learns how to predict residual of $t+1$ given M_t and S_t
 - Uses DDPM with EDM framework

EDM

- Noise scheduling: Exactly how much noise is injected or removed at any given moment.
- Preconditioning: How the inputs and outputs are **scaled** so the neural network doesn't mathematically choke on extreme values.

EDM

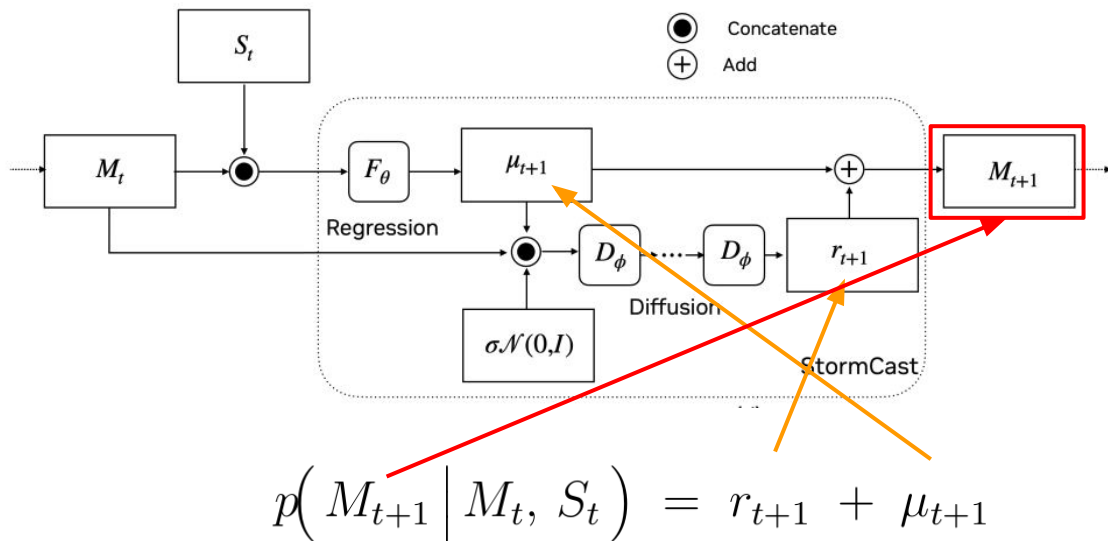
- The Solver:



Higher-order solvers

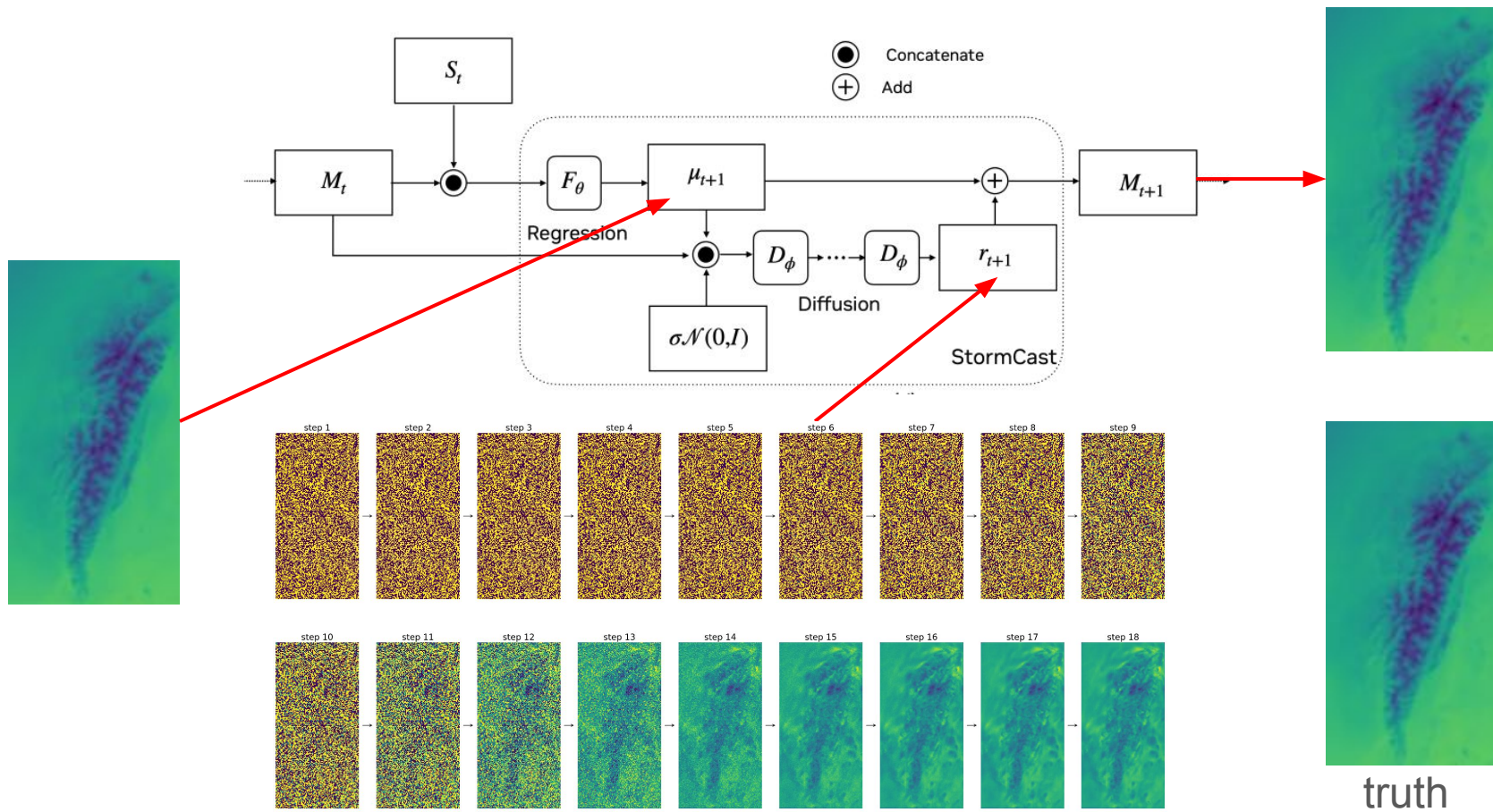
- Clever sub-steps →
higher accuracy,
higher cost
- Empirically, 2nd order **Heun method** strikes best balance

StormCast (Nvidia, 2024), collaboration with NCDR

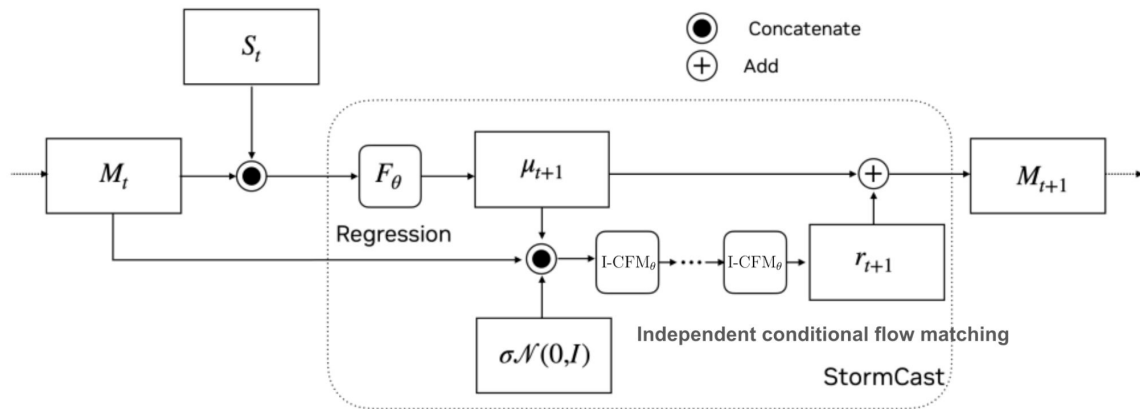


M_{t+1} is obtained by combining deterministic prediction from regression model and residual from stochastic diffusion model.

StormCast (Nvidia, 2024), collaboration with NCDR

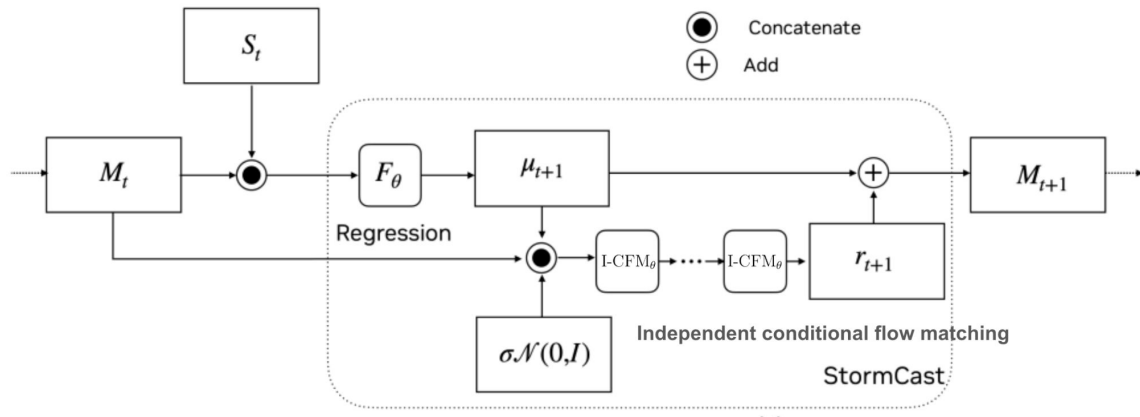


Replacing diffusion in StormCast with I-CFM



$$\frac{d}{dt} x_t = v_t(x_t), \quad x_0 \sim p_0, \quad t \in [0, 1]$$

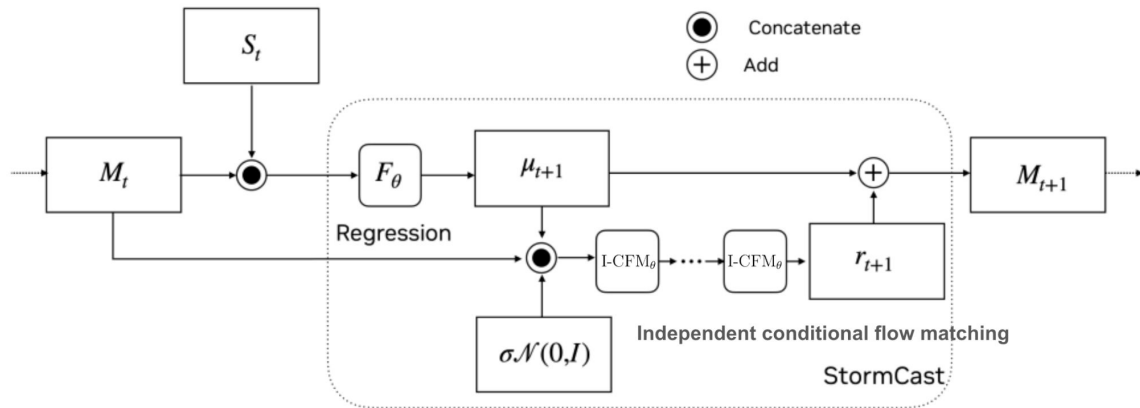
Replacing diffusion in StormCast with I-CFM



$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ x \sim p_t(x)}} \left[\left\| v_\theta(x, t) - u_t(x) \right\|_2^2 \right]$$

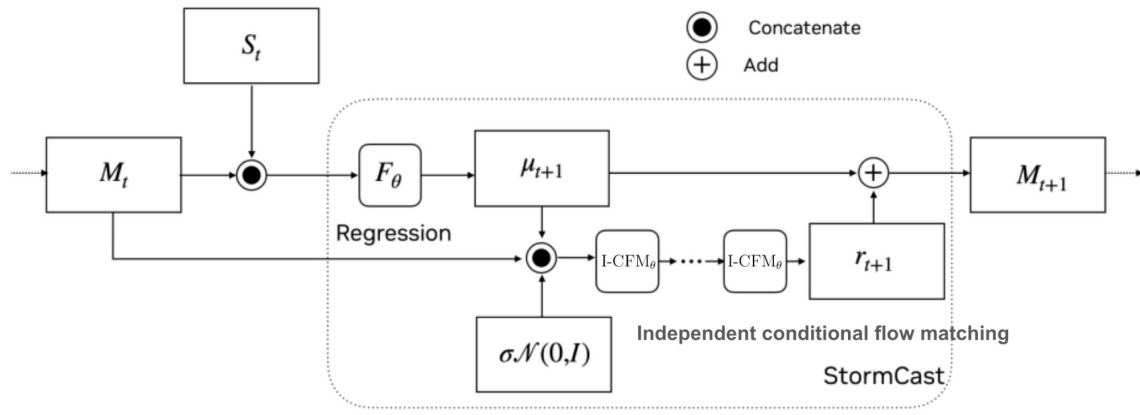
$\underbrace{p_t, u_t}_{\text{marginal — intractable}}$

Replacing diffusion in StormCast with I-CFM



$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ (x_0, x_1) \sim q \\ x \sim p_t(x | x_0, x_1)}} \left[\left\| v_\theta(t, x) - u_t(x | x_0, x_1) \right\|_2^2 \right]$$

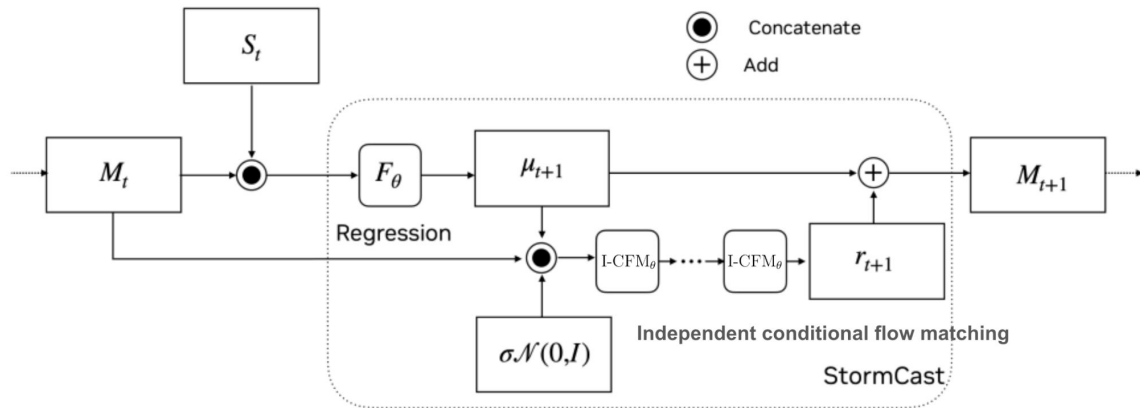
Replacing diffusion in StormCast with I-CFM



$$x_t = (1 - t) x_0 + t x_1,$$

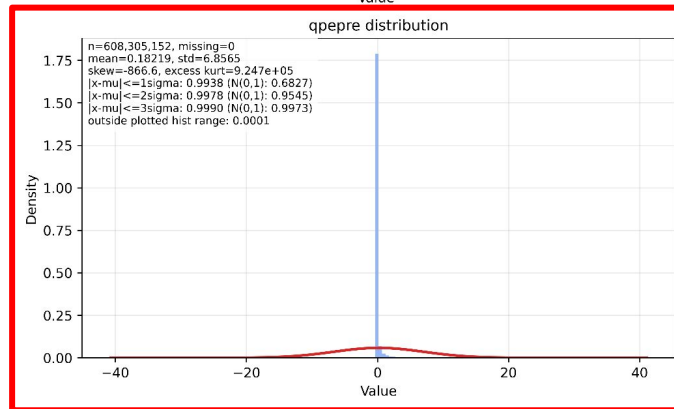
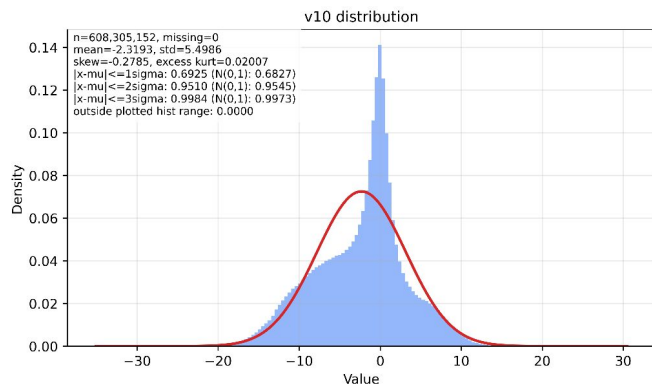
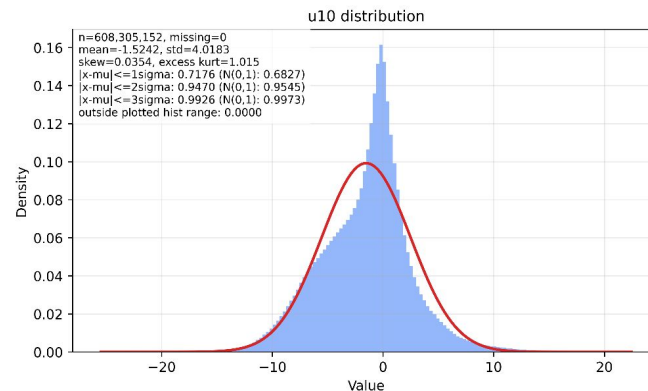
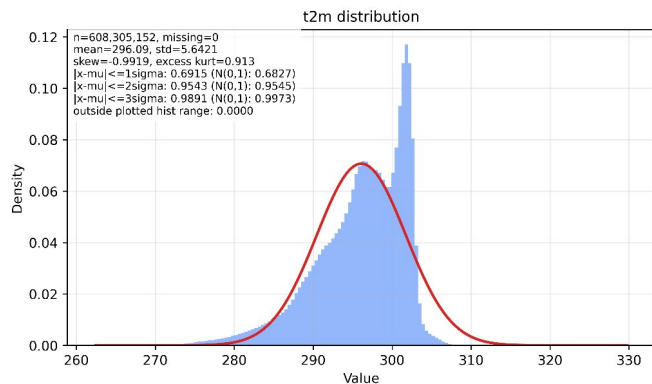
$$u_t(x_t \mid x_0, x_1) = x_1 - x_0$$

Replacing diffusion in StormCast with I-CFM



$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ x_0 \sim \mathcal{N}(0, I) \\ x_1 \sim q, \varepsilon \sim \mathcal{N}(0, I)}} \left[\left\| v_\theta(x_t, t, c) - (x_1 - x_0) \right\|_2^2 \right] \quad c = (M_t, \mu_{t+1}, I)$$

RWRF & Qpepre distributions



Log1p on Qpepre distributions

- MSE loss cares about absolute difference
- Predicting 0.1 mm/h instead of 1 mm/h (10× error) contributes the same $0.81 \text{ mm}^2/\text{h}^2$ to the loss as predicting 100 vs 100.9 mm/h (0.9% error). The model is told both mistakes are equally bad.
- log1p makes the loss measure log-ratios ($\log_{1p}(1) - \log_{1p}(0.1) \approx 0.60$ vs $\log_{1p}(100.9) - \log_{1p}(100) \approx 0.009$), so the gradient is concentrated where the physics actually differ.
- Perform $\log_{1p}$, $\ln(1 + x)$ where x is qpepre before training.
- Inverse with $e^p - 1$ for output, where p is transformed qpepre.

Cropping of domain size to fit Qpepre mismatch

- MSE loss cares about absolute difference
- Predicting 0.1 mm/h instead of 1 mm/h (10× error) contributes the same $0.81 \text{ mm}^2/\text{h}^2$ to the loss as predicting 100 vs 100.9 mm/h (0.9% error). The model is told both mistakes are equally bad.
- $\log_{10}$ makes the loss measure log-ratios ($\log_{10}(1) - \log_{10}(0.1) \approx 0.60$ vs $\log_{10}(100.9) - \log_{10}(100) \approx 0.009$), so the gradient is concentrated where the physics actually differ.
- Perform $\log_{10}$, $\ln(1 + x)$ where x is qpepre before training.
- Inverse with $e^p - 1$ for output, where p is transformed qpepre.

$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ x_0 \sim \mathcal{N}(0,I) \\ x_1 \sim q, \varepsilon \sim \mathcal{N}(0,I)}} \left[\left\| v_{\theta}(x_t, t, c) - (x_1 - x_0) \right\|_2^2 \right] \quad c = (X_{t-1}, M_t, I)$$

Replacing diffusion in StormCast with I-CFM

The diagram illustrates the StormCast architecture, which replaces a diffusion process with Independent Conditional Flow Matching (I-CFM). The process starts with an input M_t and a state S_t . These are concatenated (indicated by a circle with a dot) and then passed through a regression block F_{θ} to produce μ_{t+1} . Simultaneously, M_t is also passed through a regression block. The output of this regression block is concatenated (indicated by a circle with a dot) with a noise vector $\sigma \mathcal{N}(0, I)$. This concatenated vector is then processed by a sequence of I-CFM blocks (indicated by a circle with a plus sign). The output of the I-CFM blocks is added (indicated by a circle with a plus sign) to the output of the regression block to produce the final output M_{t+1} . The entire process is labeled "Independent conditional flow matching" and "StormCast".

$$\mathcal{L}_{\text{I-CFM}}(\theta) = \mathbb{E}_{\substack{t \sim \mathcal{U}(0,1) \\ x_0 \sim \mathcal{N}(0,I) \\ x_1 \sim q, \varepsilon \sim \mathcal{N}(0,I)}} \left[\left\| v_{\theta}(x_t, t, c) - (x_1 - x_0) \right\|_2^2 \right] \quad c = (S_t, M_t, I)$$